

Dayflow HRMS

Implementation Decisions & Integration Rules

Internal Engineering Specification

1 Purpose of this Document

This document defines the final technical and architectural decisions required before implementation of the Dayflow Human Resource Management System (HRMS).

It complements the existing SRS and does not redefine functional requirements. Its purpose is to remove ambiguity for backend and frontend developers before coding begins.

2 Scope Compliance with SRS

All features implemented must strictly come from the provided SRS document.

No features outside the SRS are to be added or removed. This includes, but is not limited to:

- Authentication and role-based access
- Employee profile management
- Attendance tracking
- Leave and time-off management
- Approval workflows
- Payroll visibility

No additional scope decisions are made in this document.

3 User Creation and Data Ownership

- A single **Admin/HR** user is seeded at system initialization.
- Employee accounts are created and managed by Admin/HR users.
- Employees do not self-register.
- All employee-related data is authoritative in the backend database.

This follows standard HRMS practices and aligns with the SRS role definitions.

4 Technology Stack

4.1 Backend

- Framework: FastAPI
- Language: Python
- Database: SQLite (MVP)
- ORM: SQLAlchemy
- Authentication: JWT (Bearer Token)

4.2 Frontend

- Framework: React
- Language: JavaScript
- HTTP Client: Axios

4.3 Architecture

- API-first architecture
- Frontend and backend are fully decoupled
- All communication occurs via defined HTTP APIs

5 Frontend–Backend Responsibility Split

5.1 Frontend Responsibilities

- Rendering UI based on API responses
- Collecting user input
- Making API calls as per the API contract
- Handling authentication tokens

The frontend does not make business decisions.

5.2 Backend Responsibilities

- Business logic and validation
- Role-based access enforcement
- Database reads and writes
- Approval and workflow decisions

The backend is the single source of truth.

6 API Communication Rules

- All data exchange uses JSON
- Frontend never accesses the database directly
- All protected endpoints require a valid JWT
- Admin-only functionality is exposed under `/admin/*` endpoints

The API contract document is authoritative and must be followed exactly.

7 Error Handling Policy

The backend follows a simple, consistent error handling strategy:

- **200 OK** – Successful request
- **400 Bad Request** – Invalid or missing input
- **401 Unauthorized** – Authentication required or invalid token
- **403 Forbidden** – Authenticated but insufficient privileges
- **404 Not Found** – Resource does not exist

All error responses are returned as JSON with a clear message field.

8 Data Policy

- No static or hardcoded business data is used
- All operational data is stored in the database
- Only the seeded Admin user exists at initialization
- All other data is created through application flows

9 Version Control Policy

- A single shared Git repository is used
- A single main branch is sufficient
- Backend and frontend code live in separate directories
- Each contributor commits their own work

10 Deferred Decisions

The following items are intentionally deferred until implementation is underway:

- Testing strategy
- Demo flow
- Hosting and deployment
- Additional documentation

These do not affect initial system correctness.

11 Final Statement

With the SRS, API contract, and this document in place, no further architectural or integration decisions are required before implementation begins.

Development may proceed.